Governed Evidence Assembly for Agent Memory

System Design, Failure Analysis, and Record Integrity in AtMem 2.3.8

Javad Taghia
AtMem.Ai Lab
https://atmem.ai

11 October 2026

Abstract

Persistent agent memory must recover useful evidence, preserve its authority, and expose when evidence is incomplete or has been altered. We present the system design of AtMem 2.3.8, which connects immutable source ranges, typed evidence views, query-derived obligations, independent candidate pools, bounded neighbourhood expansion, coverage-aware packing, and canonical revalidation at delivery. A separate integrity mechanism reconciles live memory records with commitments in the audit chain. The primary contribution is an inspectable contract between evidence assembly and governed delivery, not a new retrieval primitive or a claim of benchmark superiority. We analyze frozen formative traces to characterize failure modes. AtMem answered 5 of 23 LongMemEval-V2 questions and passed 9 of 30 DolphinBench tasks; an evaluator-verified-evidence condition answered 19 of 23 LongMem questions, exposing a large unresolved evidence-delivery gap. Because these subsets were repeatedly inspected during system iteration, the utility results cannot estimate generalization or support significance claims. Local Mem0 and AgentRunbook-R runs are retained only as configuration-specific diagnostics, not competitive rankings. In a separate third-party replication, the AGMI benchmark maintainer ran the published AtMem wheel and observed audit reports for 8 of 9 fixed storage-attack classes with the chain alone and 9 of 9 with an external checkpoint. This establishes conformance on those cases, not universal read-path enforcement. We provide source-to-claim mappings, exact artifact identities, explicit non-claims, standard conditional analyses used as design checks, and a frozen confirmation protocol whose outcomes are not reported here.

Keywords: agent memory; evidence coverage; context assembly; memory governance; audit integrity; LongMemEval-V2; DolphinBench.

1 Introduction

An agent asked to send a project update needs more than a relevant memory. It needs the correct recipient, the applicable date, the actual project outcome, and any qualification that changes what it is permitted to say. A retrieved passage can mention the project yet omit the outcome. A stored address can be exact yet belong to another workspace. An audit log can be internally valid while the memory table it supposedly explains has been modified. These are different failure modes, and improving nearest-neighbour retrieval alone cannot resolve all of them.

AtMem treats memory as a governed source of context. Its purpose is to preserve reusable evidence and control its delivery across agent hosts. The 2.3.7 release already included scope checks, lifecycle management, context receipts, optional semantic and graph retrieval, and execution continuity. Version 2.3.8 extends this foundation with a new evidence representation and assembly path. It asks not only which memories resemble a request, but which retained source ranges cover the request's declared information needs, whether they fit the context budget, and whether they are still eligible at delivery time.

The practical motivation came from failure. An early 14-question LongMemEval-V2 run returned an average of 21,539 memory-context tokens and answered no questions correctly. The corresponding no-retrieval arm also answered none. More available text had not produced useful evidence. Subsequent engineering focused on preserving exact state, retaining action neighbours, separating comparison sides, and avoiding context dominated by transport details or loosely related sources. Because those changes were guided by inspected examples, later scores on the same partition are diagnostic observations rather than confirmation evidence [2, 3].

The paper makes three contributions. First, it documents an architectural transition using the actual 2.3.7 and 2.3.8 source tags, rather than describing the previous system as an ungoverned vector database. Second, it specifies the contract connecting source evidence, query obligations, packing, authority, and final delivery, including where implementation heuristics fail to satisfy the idealized model. Third, it provides a reproducible failure analysis: formative utility traces, a separately replicated integrity conformance result, exact configuration boundaries, and a confirmation protocol that was frozen before those confirmation outcomes were observed. The paper does not claim state-of-the-art performance, a significant benchmark advantage, or completed causal ablations.

How to read this paper. Sections 3–4 explain the system from the architecture down to its data flow. Section 5 identifies the contribution relative to prior systems. Section 6 states conditional guarantees. Sections 7–8 distinguish measurement from interpretation. The appendices map claims to code and list the evidence needed for inde-

Claim class	Supported here	Explicitly not claimed
System design	Source-linked obligations are connected to packing, authority checks, canonical reload and receipts in the inspected 2.3.8 source	That every host or legacy read path enforces the receipt
Formative utility	Exact counts and failures on two inspected subsets	Generalization, significance, state of the art, or strongest-baseline ranking
Integrity	Third-party AGMI-maintainer replication on nine fixed attack classes	Complete lifecycle coverage, universal blocking, or security certification
Causal mechanism	Testable hypotheses and a frozen ablation matrix	An effect size for any individual component
Confirmation	Pre-existing disjoint confirmation partitions and a frozen execution protocol	Confirmation outcomes; none are reported in this paper

Table 1: Submission claim boundary. The manuscript treats absent evidence as absent rather than converting planned work into a result.

pendent reproduction.

2 Related Work and Positioning

Retrieval and persistent memory. Retrieval-augmented generation combines parametric generation with external information retrieval [6]. Generative Agents connects stored observations, reflection, and planning [12]. MemGPT manages information across memory tiers to work beyond the immediate context window [11]. These establish that external state and selective context construction are substantial prior art. AtMem does not claim to invent external memory, hierarchical memory, or the separation of storage from generation.

Mem0. Mem0 extracts, consolidates, and retrieves salient conversational information; its paper also studies a graph-based variant [4]. Its published architecture is therefore richer than the raw-ingestion configuration used in our comparisons. Our local arm runs the genuine OSS engine, but disables inference and substitutes deterministic token-hash embeddings. A win against that configuration cannot establish a win against default inferred memories, learned semantic embeddings, graph memory, or the hosted service. The relevant comparison in this paper is explicitly configuration-specific.

AgentRunbook-R. LongMemEval-V2 introduces AgentRunbook-R, which uses separate pools for raw observations, events, and strategy knowledge, and AgentRunbook-C, which gathers evidence through a coding agent over stored trajectories [16]. Typed pools and knowledge-specific retrieval are consequently not novel by themselves. Our closest architectural comparison is with AgentRunbook-R: AtMem adds explicit evidence obligations and an independently checked delivery boundary around its own formation and packing pipeline. We do not evaluate AgentRunbook-C in the final comparison and infer no result for it.

Evaluation of memory. LoCoMo examines long conversational histories [8]; LongMemEval separates indexing, retrieval, and reading and evaluates multiple memory abilities [15]. DolphinBench tests whether remembered information produces correct tool actions, rather than only verbal recall [5]. The present work uses formative subsets of LongMemEval-V2 and DolphinBench. Historical LoCoMo and other AtMem results are not relabelled as 2.3.8 results. Work on long-context utilization also shows that the placement of useful information can affect performance [7]; that motivates attention to packing without proving our packing method is better.

Integrity and coverage. Secure audit logging has long distinguished protected evidence from attacker-controlled storage [13]. AGMI operationalizes storage-edit tests against memory systems [1]. Our contribution here is the specific record-to-audit repair and its reproduction, not a new cryptographic primitive. Likewise, weighted evidence coverage has a classical submodular formulation [10]. We use that formulation to explain complementary selection, while making no claim that the complete implementation inherits a classical greedy approximation guarantee.

3 Architecture: What Changed Between 2.3.7 and 2.3.8

Figure 1 gives the high-level comparison. The release tags resolve to `cb1cf0f` for 2.3.7 and `267c5b3` for 2.3.8. Architectural statements below are source-derived; benchmark rows retain their own candidate identities.

3.1 The 2.3.7 foundation

The legacy `recall()` path retrieves bounded active record candidates through full-text search and can add graph-nominated records. It ranks and blends candidates, filters exclusions, and returns a limited result set. Semantic indexes and other retrieval interfaces also existed. The architecture already separated derived search structures

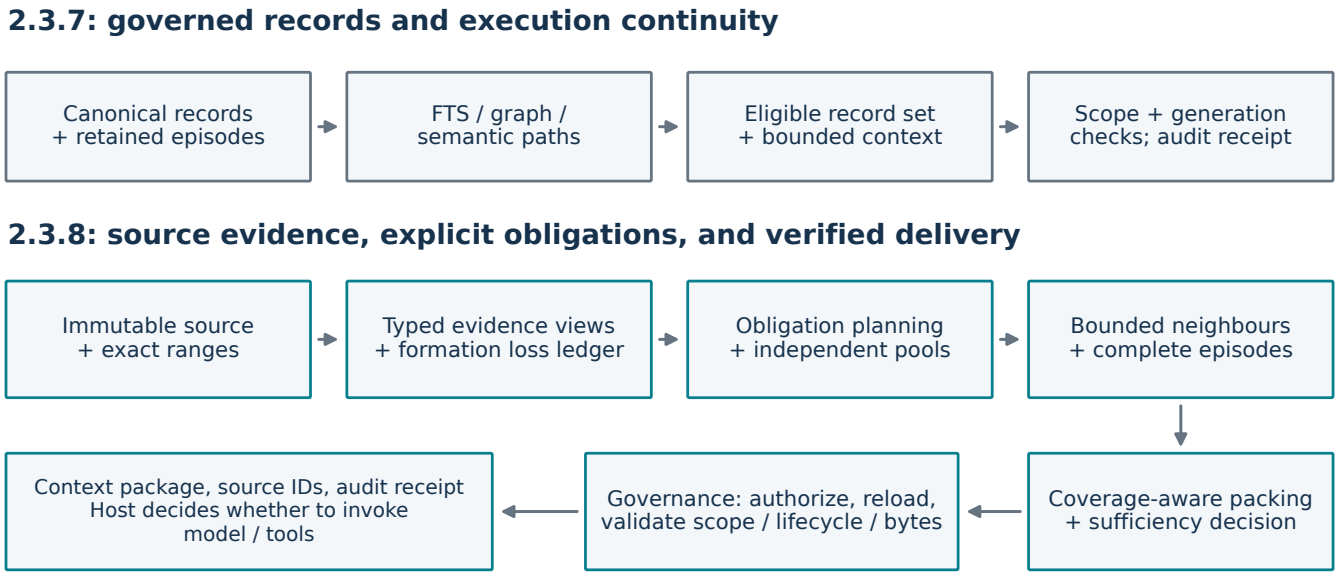


Figure 1: Architecture evolution. The upper path summarizes the 2.3.7 record/context interfaces; it does not depict every legacy retrieval path. The lower path adds an explicit source-to-obligation-to-context representation. Scope checks, generation validation, canonical reload, audit receipts, and the checkpoint API have predecessors in 2.3.7.

from canonical records.

The `prepare_context_v1()` interface loaded a candidate set bound to a subject, agent, and workspace. It checked expiry and memory generation, rejected record IDs outside the eligible set, reloaded canonical records, and excluded inactive records. Its serializer appended records within a character budget and recorded accepted IDs and a context digest. Thus, canonical reload and scoped delivery are inherited mechanisms, not claims of novelty for 2.3.8.

Execution continuity addressed another problem: preserving operation identity, receipts, recovery state, and inspection across interrupted work. That infrastructure remains valuable, but continuity does not guarantee that a later query retrieves the complete evidence needed to perform a new task. The new context engine addresses that complementary problem.

3.2 The 2.3.8 representation and control path

Version 2.3.8 introduces immutable source episodes and parts, exact ranges within those parts, and rebuildable generations of typed evidence units. A formation receipt distinguishes processing completion from representation completion and records represented and lost ranges. A query plan describes evidence obligations. Retrieval nominates evidence independently across typed pools; bounded expansion adds the local context necessary to interpret a hit. Packing then selects a feasible evidence package rather than truncating a global ranking without regard to missing needs.

The governance service is outside the selection engine. It authorizes a request, gives the engine a manifest, validates the returned IDs and receipt, reloads the exact selection, checks scope, lifecycle and generation, applies the final byte ceiling, and records an audit event. A sufficient selection whose required evidence does not survive packing is downgraded. In this service, non-sufficient status yields an empty model-context string, while the receipt explains the missing or conflicting evidence.

3.3 Deployment profiles and trust boundaries

The implementation defines three profiles: `legacy-control` for the inherited path, `context-fast` for deterministic assembly, and `context-navigate` for model-assisted navigation. Navigation requires a model. These are implementation profiles, not three separately scored winners. The final utility results are attached to the recorded candidate arm, not automatically to every profile.

Activation is explicit. Profile state tracks local qualifi-

Dimension	AtMem 2.3.7	AtMem 2.3.8 addition or change
Canonical basis	Governed records and retained episodes	Immutable source parts and exact ranges supporting typed views
Retrieval	Bounded lexical, optional graph and semantic paths	Obligation-specific nominations; typed pools; bounded source neighbours
Context assembly	Eligible record serialization under a budget	Complementary coverage and contiguous action episodes; packing-aware status
Governance	Scope, expiry, generation, canonical reload, audit	Selection/receipt consistency; exact evidence IDs; final canonical byte check
Completeness	Existing relevance and policy decisions	Formation loss receipts; required, covered and missing obligations
Integrity	Audit-chain verification and external checkpoint API	Whole-row commitments and reconciliation against live records
Hosts	Existing memory and continuity adapters	Reversible native Hermes setup, shadow/active and explicit sharing choices

Table 2: Verified architectural delta. "Addition" does not imply the corresponding general concept was absent from the wider literature.

cation, active and shadow profiles, generation, and sampling. The default-state function can retain legacy control with a shadow candidate on upgrade, and permits a qualified fast profile for a fresh installation. Hermes setup defaults to an isolated scope and shadow mode. Active recall and cross-agent sharing require explicit choices. Optional AtBot ranking or proposals and AtFlows observations do not grant those components authority to mutate canonical memory [14].

4 Technical Mechanisms

4.1 Immutable sources, typed views, and loss accounting

Let an episode contain ordered source parts $s_1, \ldots, s_m$. We represent a source range as

$$r = (\text{source}, \text{part}, a, b, h_s),$$

where a, b identify a bounded span and h_s binds the retained source. An evidence unit points to one or more such ranges and carries its kind, generation, and retrieval projection. Its identity does not replace the source's authority.

The store rejects reuse of an existing source identity with different source bytes. It creates source parts once and builds views over them. Deterministic formation retains raw ranges and adds evidence shapes for facts, entities, procedures, rules, transitions, gotchas, and premises. Additive extraction can augment these views, but an extracted claim needs grounding and applicable admission checks. Processing every input is insufficient to claim that every relevant fact has become retrievable.

The formation receipt therefore distinguishes *processing complete* from *representation complete*. It records ranges represented, ranges lost or withheld, unit counts, rejections, and storage accounting. A source retained but not represented is a diagnosable formation gap. A source refused by governance is a different outcome from a source silently dropped by an extractor. Neither becomes a successful answer merely because ingestion returned without an exception.

This design also helps remove duplication. Earlier Dolphin diagnostic checkpoints occupied about 1.028 billion bytes; a subsequent compact V3 attempt occupied 245.3 million bytes, about 76.1% less. That attempt still failed its removal diagnostic because of duplicate nominations and non-atomic targets. The storage improvement is historical diagnostic evidence, not a final matched storage-efficiency result or a claim of superior utility.

4.2 Query obligations and routing

A plan contains bounded, explicit information needs derived from the request. Examples include subject–relation–value, ordered steps, before–action–after, premise checking, and condition–action evidence. A user requesting a comparison needs both sides; a user asking what changed needs evidence for the transition, not just two nearby timestamps.

The planner is concrete heuristic software. It uses request clauses, lexical cues, identifiers, dates, and action-content facets. It strips known host scaffolding so that a tool schema does not become a fictitious memory obligation. It bounds decomposition; for example, the inspected clause splitter retains at most four distinct clauses. These choices improve tractability but can miss implicit requirements.

For addressed actions, the planner separates content needs from the delivery address. Otherwise an exact email address can dominate retrieval even when the needed outcome appears in a different source episode. Content facets remain nominations rather than new mandatory obligations. This distinction matters: adding every query expansion as a required fact would cause unnecessary blocking.

No claim of general semantic understanding follows from this planner. The source includes explicit workflow patterns, including ServiceNow Problem/Incident distinctions in option-field packing. These are application-specific heuristics introduced during system iteration. They consume request and source text, rather than evaluator answers at runtime, but still create adaptation risk and must be evaluated on unseen environments.

4.3 Independent pools and bounded evidence expansion

The V3 pool definitions include raw state, transition, fact, entity, procedure, rule, gotcha, and premise. The inspected default candidate budget permits 32 nominations per pool and 200 total units. Candidate limits are separate from the delivered-context budget. More nomination breadth does not imply permission to deliver more bytes.

The inspected fast path uses indexed lexical candidates and intent-specific ordering and grounding. Other product retrieval facilities include semantic and graph paths. The release description names all of these capabilities; that description is not evidence that every signal was active in the final benchmark arm. We therefore avoid assigning measured gains to a neural reranker, graph algorithm, or learned embedding that was not isolated in the run.

Once a head candidate is found, bounded source neighbours recover the condition, action, outcome, value tail, or identity field that makes it useful. Exact date and identifier nominations help select the correct episode. Indirect references to the usual recipient can retain an adjacent source-backed address. Selected sources may contribute related ranges without each range independently outranking every unrelated candidate.

Consider an illustrative request: "Email the integration contact the final outcome of the September review and explain the remaining restriction." Retrieving the contact directory alone finds the destination but not the outcome. Retrieving a review summary without the restriction produces an incomplete action. AtMem's intended behaviour is to cover recipient, outcome, and restriction, preserve their provenance, and block or withhold context if its required evidence is missing. This example illustrates the mechanism; it is not a fabricated benchmark trace.

4.4 Complementary packing and episode continuity

The packer prioritizes required evidence, complementary field coverage, and source groups relevant to addressed actions. Within a selected action episode it preserves source order so that trigger, action, result, and qualification remain interpretable together. This addresses a failure in which the right source was retrieved but fragmented around unrelated recipients or caveats.

One implementation helper greedily prefers new option-field coverage per byte. Another orders action source groups and preserves their contiguous ranges. The implementation is consequently more specific than a generic "top-k" selector and less general than a globally optimal constrained coverage solver. The service subsequently uses the IDs that actually survived packing. It does not reload excluded candidates and accidentally reintroduce text that was deliberately removed to satisfy the budget.

4.5 Sufficiency is a typed decision, not a calibrated probability

The actual status vocabulary includes `sufficient`, `partial`, `conflicted`, `contradicted`, `stale`, `not_found_within_budget`, and `withheld_by_policy`. Required obligations are covered by matched evidence IDs, minus obligations withheld because their source episode is incomplete. Missing required IDs prevent a sufficient result.

Conflict and contradiction detection are bounded heuristics. The inspected code checks negative language for premise obligations and distinct numeric or channel values for certain approval-sensitive questions, with correction cues. This does not establish general contradiction resolution. A receipt saying "sufficient" means the implemented coverage rule accepted the declared obligations; it does not mathematically certify that an arbitrary natural-language answer is true.

Packing can invalidate sufficiency: retrieving complete evidence does not help if required material is excluded from the final package. The service downgrades a sufficient decision when packing is incomplete, records `context_budget_exhausted`, and propagates the actual selected IDs. This coupling between the decision and the delivered package is a central integration contribution.

4.6 Canonical revalidation and host responsibility

The selection engine operates under an authorized manifest. The governance service rejects duplicate IDs, unauthorized selections, sufficiency evidence outside the selection, mismatch between plan obligations and the receipt, and undeclared external egress. It reloads selected canonical evidence and verifies exact order, scope, generation, and active lifecycle. The context digest and audit event identify what the service prepared.

This protects the context boundary against an erroneous or overreaching selector under the stated authorization assumptions. It does not imply that a host will obey the receipt, that arbitrary tools cannot bypass the host adapter, or that earlier context already observed by a model can be erased. The Dolphin adapter's pre-action gate is one explicit consumer of the status. A different integration needs its own enforcement and observation tests.

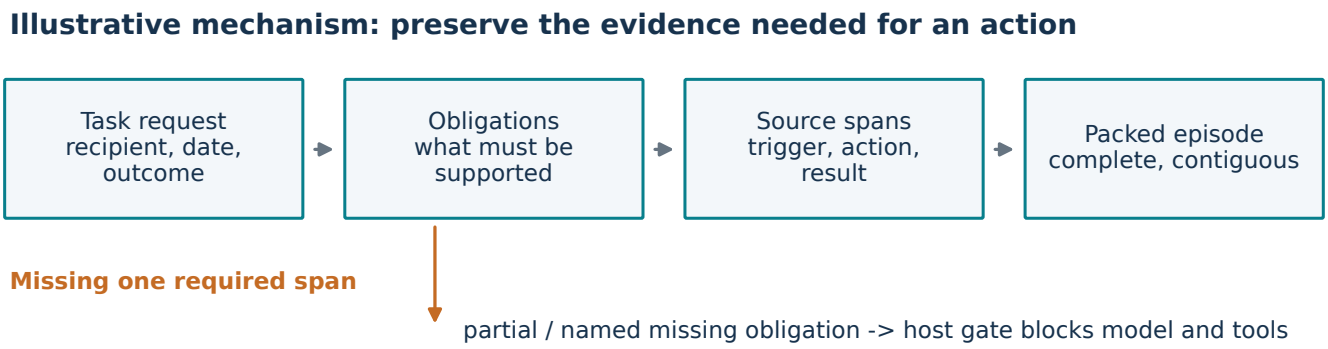


Figure 2: A conceptual request-to-evidence path. Runtime obligations come from the request; evaluator requirement annotations are reserved for diagnostics. A host must explicitly enforce the resulting status for missing evidence to prevent model or tool invocation.

5 Contribution Scope and Mechanism Hypotheses

The simplest description of the proposed contribution is *evidence obligations connected to governed delivery*. It is an engineering composition with inspectable contracts. It is not a claim to have invented typed memory, source attribution, graph retrieval, or a hash chain.

Novel integration relative to the earlier AtMem. The old context boundary proved eligibility of a selected record set. The new path adds an explicit explanation of what information is needed, which source ranges represent it, which parts are missing, and whether that support survived the packing budget. It extends existing canonical revalidation to typed evidence and selection receipts. This allows downstream hosts to distinguish evidence absence from a retrieval score below a threshold.

Relation to AgentRunbook-R. AgentRunbook-R is the closest baseline for typed evidence retrieval. Our distinction is not "structured versus unstructured memory." It is the additional connection between obligations, formation losses, source-range packing, and independently validated delivery. The final advantage over its local configuration is one question. That result is too small to isolate the value of the additional contracts or to establish a ranking against its published configuration.

Relation to Mem0. The evaluated raw Mem0 arm returned remembered strings through its real search API. AtMem's path additionally organizes and packs source evidence around a request's explicit needs. This is a plausible explanation for greater utility on tasks needing several related facts. But the comparison also changes representation and retrieval policy while using a constrained embedding route. The observed difference cannot be attributed solely to the memory abstraction, and it says nothing about the best achievable Mem0 configuration.

Mechanism hypotheses, stated without causal credit. The formative traces suggest three hypotheses: preserving exact values and workflow identity increases availability of answer-bearing evidence; bounded expansion recovers necessary conditions and outcomes; and packing complete episodes reduces reader omissions. Historical diagnoses and code changes support these as explanations worth testing. No completed factorial ablation assigns an independent effect size to any of them. Governance and integrity address reliability dimensions that were not isolated as causes of the clean-data accuracy changes.

The claim defended in this paper. AtMem 2.3.8 implements a source-linked evidence-assembly contract that can expose named formation, coverage, packing and governance failures before delivery. The inspected traces demonstrate that the contract is executable and diagnostically useful, but they do not establish that the system is more accurate than another memory architecture. The local competitor rows are retained to document the engineering environment and to prevent selective omission of completed runs. They are not used to rank products or assign causal credit.

6 Design Invariants and Conditional Analysis

The following statements are design-review instruments. The coverage result is a standard submodular-coverage fact, the authority and packing statements follow from explicit preconditions, and the rollback argument is a standard indistinguishability observation. We do not present them as algorithmic novelty. Their purpose is to identify the assumptions that implementation and evaluation must

Question	AtMem 2.3.8	Local Mem0 OSS arm	Local AgentRunbook-R arm
What is stored?	Source episodes, exact ranges, derived typed units	Raw trajectory/state or dated-message records; inference disabled	Upstream runbook memory structure with model-assisted formation
How is evidence selected?	Query obligations, typed pools, grounded heads, bounded neighbours	Qdrant search over hash embeddings	Upstream runbook retrieval with local hash-embedding route
How is context assembled?	Required coverage, complementary ranges, contiguous action sources	Returned memory strings	Runbook context gathering under local runner configuration
What is checked at delivery?	Manifest, scope, generation, lifecycle, selected IDs, byte ceiling, audit	Comparator adapter does not add AtMem's receipt protocol	Comparator adapter does not add AtMem's receipt protocol
What is measured here?	Both utility samples; AGMI reproduction	Both utility samples; separate historical AGMI row	LongMemEval-V2 sample only

Table 3: Configuration-specific comparison, not an exhaustive product feature matrix. Absence from an adapter does not establish absence from a competitor's other products or deployments.

test; they are not formal verification of the Python code or evidence of benchmark superiority.

6.1 Authorized evidence and coverage

Let U be the retained evidence units, q a request, and $A(q) \subseteq U$ the units allowed by the current authority, lifecycle and generation checks. Let $O(q) = \{o_1, \dots, o_m\}$ be declared obligations. Define $g(u, o) \in \{0, 1\}$ as a fixed evidence-grounding relation. For nonnegative weights w_o, define

$$F(P) = \sum_{o \in O(q)} w_o \, \mathbf{1}[\exists u \in P : g(u, o) = 1], \qquad P \subseteq A(q). \tag{1}$$

Packing seeks high coverage under a context cost $c(P) \leq B$, subject to required obligations and eligibility. This objective is an explanatory abstraction. Actual coverage labels are heuristic, costs include serialization overhead, and source grouping changes which candidate combinations are feasible.

Proposition 1 (Diminishing returns of fixed obligation coverage)**.** *For fixed $A(q)$ and g, F is monotone and submodular. If every unit has equal cost and feasibility is only $|P| \leq k$, classical greedy coverage obtains at least $(1 - 1/e)$ of the optimal coverage.*

Proof. Adding a unit cannot remove a covered obligation, proving monotonicity. For $P \subseteq Q$, the obligations newly covered by a unit outside Q form a subset of those it newly covers outside P. Nonnegative weights preserve this inequality, proving diminishing returns. The equal-cost cardinality case then satisfies the assumptions of the classical greedy bound [10]. □

The production packer includes unequal byte costs, mandatory heads, grouped sources and heuristic ordering. We explicitly do not transfer the $1 - 1/e$ guarantee to it. If support requires a conjunction of several units, the binary per-unit relation is also insufficient; a hyperedge or structured-obligation model is needed, and the resulting objective need not be submodular.

Proposition 2 (Conditional authority confinement)**.** *Assume the authorizer returns exactly the permissible IDs for request q; canonical reload and lifecycle checks are sound; and no unauthorized change occurs between validation and serialization. If preparation returns successfully and serializes only its reloaded selected units, every delivered unit lies in $A(q)$.*

Proof. The service first checks selection containment in the authorized manifest. Reload must return the exact selected IDs and order. It then checks each unit's scope, generation, and lifecycle. Serialization uses only those units. Any delivered unit outside $A(q)$ would contradict at least one assumed sound check. □

This proves an output-membership property, not timing noninterference, correct authorizer policy, or protection against a host bypass. Transaction and concurrency behaviour must preserve the stated assumptions.

Proposition 3 (No loss of declared support at a correctly enforced packing boundary)**.** *Assume $O(q)$ contains all required obligations, g is sound, and the emitted sufficient status is permitted only if every $o \in O(q)$ is supported by the final packed set P. Then sufficient delivery entails declared evidence coverage within the budget.*

Proof. The status precondition gives a supporting unit in P for every required obligation. The packer and final serializer check $c(P) \leq B$. Both claims follow directly. They do not imply that the reader interprets the evidence correctly. □

The assumptions are deliberately strong. The implemented planner may omit needs, grounding may be inaccurate, and contradiction handling is incomplete. Removal

controls test selected instances of this boundary; they do not establish universal semantic sufficiency.

6.2 Integrity binding and rollback

Let $\phi(r)$ be a canonical projection of a memory record and $d_r = H(\phi(r))$ its committed digest. The projection includes identity, subject, content, source fields, trust tier, time, confidence, scope, status, supersession, fact key, and raw payload. A protected audit event commits to d_r. Verification must reconcile live records against events, rather than checking only links between events.

Proposition 4 (Detection for an unchanged, committed record)**.** *Assume a trusted audit commitment d_r cannot be replaced, H is second-preimage resistant, the verifier checks the live record against d_r, and no authorized lifecycle exemption applies. Changing any field included in $\phi(r)$ is detected except with negligible cryptographic probability.*

Proof. An altered projection $\phi(r') \neq \phi(r)$ can pass only if $H(\phi(r')) = d_r$. This is a second preimage under the assumptions. Removing a required committed row is detected by existence reconciliation rather than a hash comparison. □

This statement is intentionally narrower than "all memory is tamper-proof." The inspected implementation leaves legacy records without commitments identifiable as legacy and skips records marked legitimately changed by certain lifecycle events. Integrity coverage for those paths must be audited separately. A valid digest also cannot establish whether originally admitted text was factually correct.

Theorem 1 (Whole-store rollback cannot be recognized from the rolled-back store alone)**.** *Suppose a verifier has no trusted state outside a writable store. If it accepts a legitimate earlier snapshot S_t, it cannot always reject the identical snapshot after an attacker replaces a later store S_{t+1} with S_t.*

Proof. Compare two worlds: the store legitimately remains at S_t, and the store advanced to S_{t+1} before being restored to S_t. The verifier receives identical trusted inputs in both worlds. Its output distribution is therefore identical. Always accepting the legitimate old state and always rejecting the rollback is impossible without another trusted distinguishing input. □

An external checkpoint containing a later chain position and hash provides such an input. Its freshness bounds the protected interval, and its storage must be outside the attacker's rollback domain. Merely hashing a checkpoint stored beside the database does not satisfy this requirement.

7 Experimental Method and Configuration Audit

7.1 Evidence scope and provenance

This paper analyzes retained reports, source, and the mounted public benchmark corpora; it does not run new paid experiments. The LongMemEval-V2 question corpus was verified against the recorded SHA-256 value. Large trajectories are not redistributed in the arXiv bundle because they remain governed by their upstream distribution and are reproducible from the pinned dataset revision. Historical aggregate counts are verified against retained compact records. The manuscript does not claim that every earlier provider response or grading trace was independently regenerated.

The stable architecture is read at tag `v2.3.8`. The utility measurements were made on pre-release candidate builds, including artifacts retaining the `2.3.8b6` version. The final Dolphin candidate is `b97d35e`. The vendor AGMI candidate wheel was built from `fdc63de`; the external AGMI maintainer performed a third-party replication from the published PyPI wheel, whose installed package matches tag `v2.3.8`. Relative to `fdc63de`, the package differs only in a compatibility comment and tested OpenClaw version string. A single stable tag was not rerun across all utility rows. We use "2.3.8 candidate" for utility and keep AGMI's published 2.3.8 identity explicit.

7.2 Samples and metric definitions

The formative LongMemEval-V2 subset uses 23 of 451 questions, or 5.10%. It retains all 14 historical pilot IDs and adds nine IDs selected by a salted hash from the remaining development partition. It is therefore an inspected nested sample, not a fresh test. The formative DolphinBench subset uses 30 of 600 tasks, exactly 5%, across three personas. Historical ingestion processed all 13,539 released sessions; the small task fraction does not mean only 5% of the history was ingested.

Crucially, both benchmark protocols had already reserved disjoint confirmation partitions before the reported iterations. LongMemEval-V2 reserves 318 question IDs under split digest `8e5d5e2d...`; DolphinBench reserves 582 task IDs under its frozen task split. No confirmation outcome is opened or reported here. The existence of those partitions improves the feasibility of a future test but does not retroactively validate the formative results.

LongMem accuracy is $\sum_i y_i/n$, with failed system executions retained in n. Dolphin task success requires the task's complete grading outcome; individual check success is a separate metric. The 97 checks are clustered within 30 tasks and are not 97 independent experimental units. AGMI reports detection outcomes for nine fixed attack classes; these are conformance cases, not an estimated probability of detecting arbitrary attacks.

7.3 Versions, models and comparator boundaries

The pinned Mem0 source declares version 2.1.0 in its `pyproject.toml` [9]. The historical AGMI Mem0 row is separately versioned 2.0.20 and must not be used to label the utility comparator. The utility adapters explicitly import from the pinned checkout, so the source revision is more informative than a possibly different installed distribution's metadata. A fresh environment lock is still needed for exact reproduction.

The LongMem Mem0 adapter uses Qdrant, a 768-dimensional signed token-hash embedding, raw trajectory metadata and state records, and `infer=False`. Its default query limit is 20 and it discards the query-image argument. The formation runner also supplies AgentRunbook-R with the local 768-dimensional hash-embedding endpoint and Qwen controller; it requires explicit paid formation for that arm. This is distinct from a separate reader-free offline AgentRunbook adapter that disables model-assisted formation. The two experimental families must not be conflated.

For Dolphin, the Mem0 adapter uses 256-dimensional hash embeddings and raw dated messages with inference disabled. The report additionally notes absent optional spaCy and fastembed extras. These details materially limit what the comparison says about Mem0. Earlier reference manifests name a learned Qwen embedding model; the inspected execution code overrides that route for these local configurations. The paper follows the concrete adapter and runner configuration, rather than treating a general manifest as evidence of a learned model actually used.

The final Dolphin reader route was Hugging Face inference providers; LongMem used the recorded RunPod reader route. Shared model identity does not imply shared provider execution across the two benchmarks. The final Dolphin report reuses a Mem0 artifact with identical task IDs, source, checkpoint, reader, and judge according to its provenance statement. Earlier Mem0 counts of 3 and 5 belong to other attempts; the final reference is 7. We retain that provenance rather than interpreting a changed aggregate as an unexplained gain by AtMem alone.

7.4 Controls and failure accounting

LongMem includes no-retrieval and evaluator-verified evidence controls. The latter supplies privileged supporting evidence to the same reader; it is a diagnostic upper reference for that controlled input, not a deployable competitor or a universal accuracy ceiling. Its success indicates opportunity to improve evidence delivery or use, but not which pipeline stage is solely responsible.

Dolphin removal controls remove evaluator-selected evidence after formation, then require a named missing obligation, no model invocation, and zero tool calls. Restoration is a positive control: an agent that always blocks must not receive a safety success. The recorded removal result is 30/30 and restored gate opening is 30/30; paired removal-and-positive controls are 29/30, with one provider failure. These controls belong to their recorded formative run and are not silently relabelled as a fresh final-candidate control run.

8 Failure Analysis on Inspected Partitions

8.1 Absolute performance and failure accounting

AtMem fails to answer 18 of 23 LongMem questions correctly and does not fully pass 21 of 30 Dolphin tasks. Six LongMem cases are classified as system failures: three reader finalization failures at the 20,000-token completion cap and three governance rejections described in the report as outside the frozen selection. That wording requires case-level reconciliation before allocating terminal blame. All six remain in the denominator; we do not remove them to increase accuracy.

8.2 Evidence versus no memory

The no-retrieval LongMem arm answers 1/23, while evaluator-verified evidence yields 19/23. AtMem product context obtains 5/23. Thus product context has four more observed successes than the no-memory control, while a 14-question aggregate gap remains to the verified-evidence condition. The central empirical observation is the unresolved gap, not the four-case increase. These are aggregate differences, not claims that exactly four specific cases were rescued or that precisely 14 retrieval errors occurred. Case overlaps and controlled stage ledgers are needed for that attribution.

8.3 Why the local competitor rows do not establish a ranking

The retained report also contains local diagnostic arms: AgentRunbook-R answers 4 of 23 LongMem questions, Mem0 OSS answers 1 of 23, and Mem0 passes 7 of 30 Dolphin tasks. These arms use constrained configurations, including hash embeddings and disabled Mem0 inference, and no AgentRunbook-R Dolphin run exists. They are disclosed for completeness in Appendix E, not promoted as a product comparison.

Let a and b be successes of two systems on the same n cases and h the number both solve. Discordant counts are $a - h$ and $b - h$. Without paired outcomes, their overlap is unknown, but is bounded by

$$\max(0, a + b - n) \leq h \leq \min(a, b). \tag{2}$$

For the conventional two-sided exact McNemar test, enumeration of all feasible overlaps gives minimum possible p values of 0.125 for LongMem AtMem versus Mem0, 1.0 for

Component	Frozen identity / configuration
AtMem stable architecture	2.3.8, commit `267c5b3`; 2.3.7 comparison at `cb1cf0f`
AtMem utility candidates	Historical 2.3.8b6 identities retained; final Dolphin code `b97d35e`
Mem0 OSS	Pinned source declares 2.1.0 at `d3891e48baa2c6e769f9cfa4003873bd6a85bc07`; raw ingestion, `infer=False`
AgentRunbook-R	Source revision `2cc8c540bdb87fe6761629b585e727e1c4704520`; no independent semantic release number recorded
Reader / agent	Qwen/Qwen3.5-9B, revision `c202236235762e1c871ad0ccb60c8ee5ba337b9a`
Judge route	`gpt-5.2-2025-12-11`, medium reasoning, where semantic grading applies
LongMem embeddings	Local `atmem/hash-bow-768-v1` route; signed token hashing, not a learned semantic encoder
Dolphin Mem0 embeddings	Local `hash-bow-256-v1`; raw dated-message ingestion
AGMI	0.6.3, commit `115493a41a7b41952f92ec07e1ea0932926e194f`

Table 4: Configuration card. "Matched" means shared case IDs and reader/judge route within the recorded comparison; it does not mean equal architecture, equal formation cost, or strongest possible settings for every system.

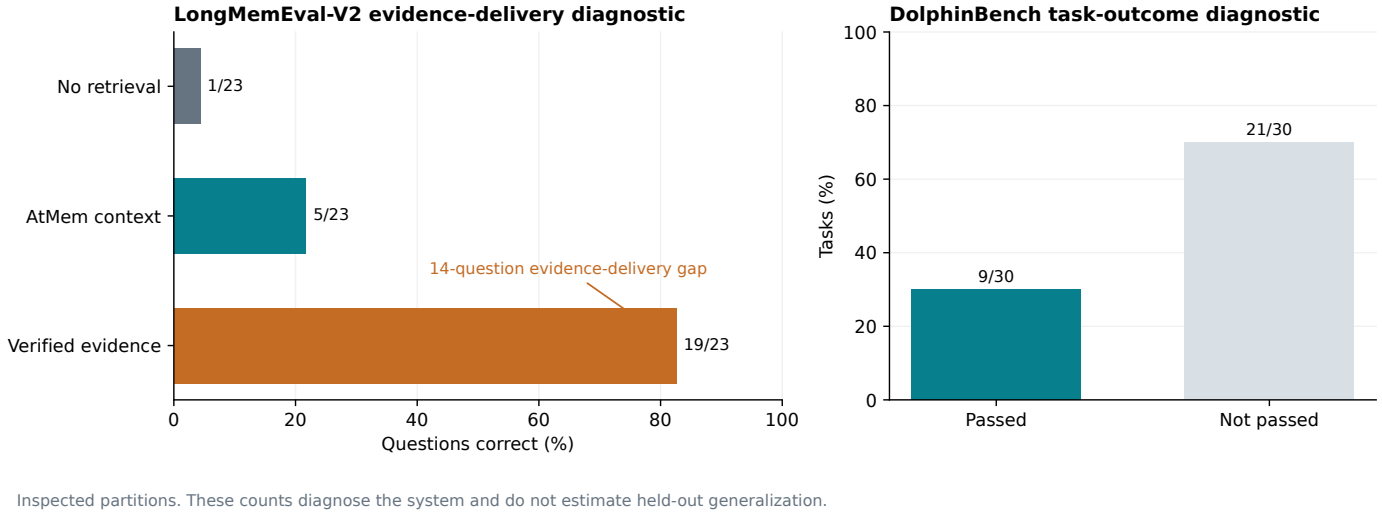


Figure 3: Formative utility diagnostics. The left panel exposes the gap between product-supplied and evaluator-verified evidence; the right reports absolute AtMem task outcomes. These inspected partitions do not estimate held-out generalization.

AtMem versus AgentRunbook-R, and 0.5 for Dolphin AtMem versus Mem0. Therefore, even the most favourable overlap consistent with the published counts would not yield significance at 0.05 under that test. The accompanying script reproduces this calculation without inventing paired data.

This is a sensitivity calculation, not a formal inferential test. Repeated inspection and constrained baseline configurations prevent interpreting these counts as an unbiased generalization estimate. We omit population confidence intervals, paired bootstrap intervals, and speed/cost superiority claims because the available evidence does not justify them.

8.4 Engineering trajectory, not causal ablation

The early LongMem pilot's 0/14 and final 5/23 use different sample sizes and evolving code; they cannot be subtracted as a clean treatment effect. A seven-question diagnostic briefly reached 4/7 after changing evidence projection and reader limits, followed by 3/7 on another confirmation. The retained negative results show why a peak intermediate score should not be selected as the release score [2].

For the fixed Dolphin formative subset, the earliest retained AtMem result is 2/30 tasks and 15/97 checks. The final result is 9/30 and 32/97 following action-evidence neighbourhood and identity-routing work. This association motivates the mechanism hypotheses, but model/provider effects, repeated attempts, and multiple code changes remain confounded. A valid ablation would freeze the same artifact, formation checkpoint, reader, provider, budgets, and repetition policy while removing one mechanism at a time.

Metric	Observed count	Diagnostic interpretation
LongMem correct	5 / 23 (21.7%)	Product context remains far below verified evidence
LongMem system failures	6 / 23	Operational reliability is part of the measured deficit
LongMem verified-evidence control	19 / 23 (82.6%)	Reader can often answer when supplied correct support
Dolphin tasks passed	9 / 30 (30.0%)	21 complete tasks remain unsolved
Dolphin checks passed	32 / 97 (33.0%)	Checks are clustered, not independent units
Dolphin system failures	0 / 30	No recorded terminal system failure in this run

Table 5: Absolute formative outcomes and controls. All selected cases, including failures, remain in the denominator.

9 AGMI: From an Intact Audit Chain to Verifiable Memory Records

9.1 The exposed gap and the repair

The pinned upstream 2.3.7 AGMI result reports none of nine attacks for the chain-only profile and one of nine for the external-checkpoint profile. This did not mean AtMem had no audit chain. It meant record-level edits could change the table read by the agent without invalidating the independently verified audit events. The checkpoint already exposed the whole-store rollback case. The fundamental repair in 2.3.8 is binding the live record projection to an audit commitment and reconciling the two.

Creation and quarantine events can carry a whole-row integrity digest. Verification collects commitments, known record IDs, and legitimate lifecycle changes, compares eligible live rows to their commitments, reports missing committed rows, and identifies non-tombstoned records with no recognized creation event. Checking content alone would miss attacks on ownership, trust, ordering-related metadata, or lifecycle state; the explicit projection broadens the bound fields.

9.2 Measured result and operational interpretation

The installed 2.3.8 wheel reports T1–T8 under both profiles: tampering, truncation, middle deletion, reordering, forging, cross-context replay, record rollback replay, and metadata tampering. T9, whole-store snapshot rollback, is accepted without an external checkpoint and reported with one. The result is 8/9 and 9/9 respectively. A historical Mem0 2.0.20 local-Qdrant row in the same pinned upstream scorecard accepted all nine; it is a separate integrity configuration, not the utility run reused as an integrity experiment.

The report records *reported* verdicts. We therefore claim audit detection, not universal blocking before every read. The checkpoint must remain trusted, outside the writable store, and be updated after genuine writes to protect those writes against rollback. Its mere existence does not protect changes made after the latest anchor.

Legacy stores and legitimate mutations are explicit limits. The inspected verifier enables whole-row reconciliation only when a commitment exists. It also skips records identified as legitimately changed by specified events. A malicious edit after such a transition may require a refreshed commitment and additional checks; the fresh seeded AGMI cases do not prove complete lifecycle coverage. This manuscript documents that boundary rather than using a 9/9 result as evidence that every persisted AtMem table or mutation path is protected.

9.3 How integrity contributes to useful memory

Integrity improves the basis on which a memory can be trusted for reuse. A source-backed recipient, date, or permission is useful only if the stored evidence still corresponds to the authorized history. Record reconciliation detects a class of silent substitutions, and an external checkpoint detects an otherwise indistinguishable old store. These support reconstruction and investigation of why a memory was supplied.

They do not establish that the evidence was true at admission, eliminate prompt injection, or improve a reader's answer on their own. The clean-data utility benchmarks contain no controlled integrity intervention. Accordingly, we claim complementary utility and integrity evidence, not that AGMI success caused the LongMem or Dolphin score increase. An appropriate future experiment would measure action correctness before and after deliberate storage attacks, with verification enforcement on and off.

10 Discussion, Limitations, and Research Agenda

10.1 Why the architecture merits continued investment

The durable value proposition is an inspectable contract across source retention, context selection, authority, and

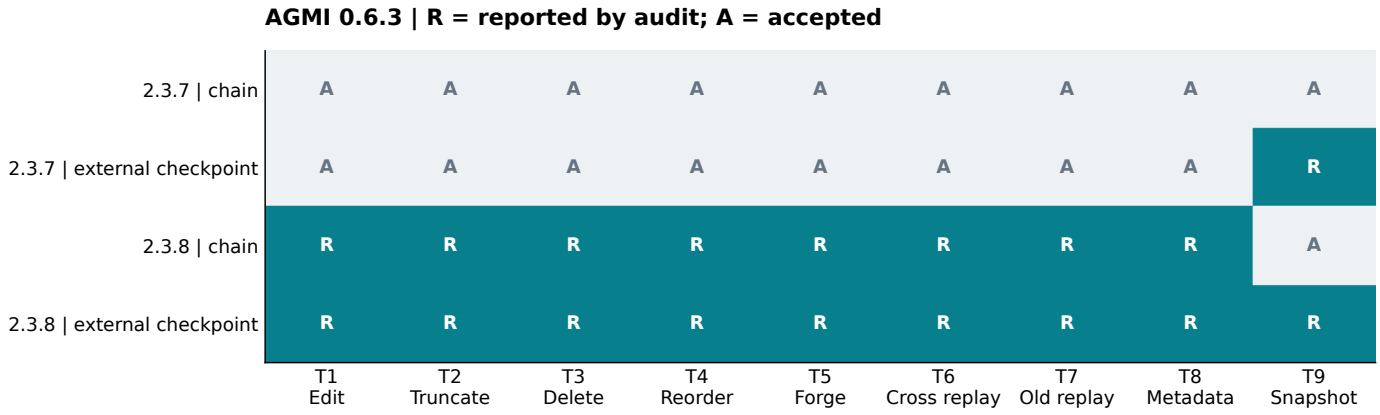


Figure 4: AGMI attack-class outcomes. "Reported" means the audit path reports a problem, not that every ordinary read API independently rejects it. The 2.3.8 rows were replicated by the external AGMI maintainer from the published PyPI wheel on Linux and macOS; the 2.3.7 rows are the retained upstream baseline.

host delivery. The retrieval engine can change without transferring canonical authority to a model or index. Source ranges make it possible to investigate selection and rebuild derived views. Missing-obligation receipts let a host choose a controlled response instead of treating a search hit as sufficient permission to act. Record commitments make part of the persisted history auditable against the live state.

These properties offer a credible reason for AtMem to compete alongside established memory systems. They are architectural reasons to invest in the system, not a prediction that the product must succeed. A deployment must still demonstrate acceptable latency, storage overhead, recall, operational reliability and host support on its own workload.

10.2 Threats to validity

The central limitation is formative adaptation. The LongMem subset contains earlier inspected questions, and the Dolphin subset was reused while adding rules and selecting candidate code. Runtime separation from evaluator labels is necessary but does not remove tuning to a benchmark's vocabulary or failure modes. Application-specific branches in the packer are especially important to disclose.

Comparator strength is another limitation. Hash embeddings omit semantic capability expected from modern learned encoders. Mem0 inference is disabled in both utility comparisons. AgentRunbook-R uses a local configuration rather than its published strongest setup. Equal question IDs and reader routes do not establish equal optimization budgets, equal formation cost, or equal modality handling. The final report's missing optional Mem0 extras further narrow its interpretation.

Operational failures confound quality. The LongMem system-failure counts differ substantially across arms. The final Dolphin run has no recorded system failures, but stochastic readers and reused comparator artifacts still require symmetric repeated trials. The observed task advantage is small, and checks within a task are dependent. No uncertainty band should disguise these facts.

Formal analysis also has limits. The planner is not complete, grounding is not a sound general theorem prover, and the source does not establish constant-time or complete cross-scope noninterference. The integrity argument assumes a trusted commitment; lifecycle exemptions and legacy rows need additional qualification. A nine-case suite cannot justify a complete security claim. Public corpora and compact receipts are available, but historical provider responses and all case-level grading traces were not independently regenerated for this manuscript.

10.3 Confirmation experiments

The repository already contains disjoint confirmation partitions frozen before the formative runs. A confirmation study must keep those outcomes sealed until the AtMem artifact, prompts, reader, judge, budgets, repetitions, exclusion rules and analysis code are hashed. It should run every compared system under both a resource-matched profile and a documented strong configuration. The strong comparison should include a learned-embedding Mem0 arm with normal extraction, preserve the current raw arm as an ablation, and reproduce AgentRunbook-R with its recommended retrieval and formation settings. AgentRunbook-R's absent Dolphin row remains absent until an actual adapter and run exist.

Component ablations should remove typed formation, obligation routing, source-neighbour expansion, episode grouping, and sufficiency gating separately. The endpoint should include evidence-set recall, exact-value retention, action success, false blocking, and reader finalization. Ablation results must retain errors and failed admissions. A repeated-trial paired analysis on held-out tasks

can then test which components contribute beyond chance and which merely fit inspected examples.

Governance qualification should examine concurrent invalidation, cross-scope selection, deletion propagation, expired evidence, host bypass, and the difference between prepared and actually delivered context. Integrity qualification should include legitimate updates followed by tampering, pre-upgrade rows, all relevant storage surfaces, stale checkpoints, and automatic enforcement modes. These extend the current record-level AGMI evidence into a broader operational argument.

11 Conclusion

AtMem 2.3.8 advances the project from governed record selection toward explicit evidence assembly: retain source ranges, form typed views, state the information required, gather complementary support, preserve it within the budget, and validate the final selection before delivery. The integration extends a substantial 2.3.7 governance foundation. Whole-row audit commitments separately address the memory-table gap exposed by AGMI.

The evidence supports a narrower, constructive claim: source-grounded assembly and memory governance can be connected through inspectable contracts that expose formation loss, missing obligations, packing loss, authority failure and record-integrity mismatch. The formative utility study reveals a large evidence-delivery gap rather than a solved benchmark, while the separately replicated AGMI cases establish limited integrity conformance. Generalization, comparative ranking and component effect sizes remain open empirical questions reserved for the frozen confirmation protocol.

Disclosure and authorship. Javad Taghia designed and implemented the system and is responsible for the claims in this preprint. Text organization, source inspection support, and figure preparation used an AI coding assistant; all quantitative claims are tied to cited artifacts rather than generated by the assistant. The utility measurements predate the manuscript and were produced by the AtMem project. The AGMI 2.3.8 rerun was performed by the external AGMI benchmark maintainer from the published wheel. This manuscript has not undergone peer review.

References

[1] AGMI contributors. Agent memory integrity: Version 0.6.3, 2026. URL https://github.com/tech4biz-yasha/agmi/tree/115493a41a7b41952f92ec07e1ea0932926e194f. Pinned upstream commit 115493a41a7b41952f92ec07e1ea0932926e194f.

[2] AtMem maintainers. Atmem benchmark development runs: 30 september 2026, 2026. URL https://github.com/aetna000/atmem/blob/v2.3.8/benchmarks/retrieval_quality/reports/benchmark-3pct-complete-20260930.md.

[3] AtMem maintainers. Frozen five-percent development benchmark results, 2026. URL https://github.com/aetna000/atmem/blob/bb05043/benchmarks/retrieval_quality/reports/five-percent-results-20261010.md. Recorded 10 October 2026; aggregate report frozen at bb05043.

[4] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. URL https://arxiv.org/abs/2504.19413.

[5] DolphinBench contributors. DolphinBench: Mapping the pareto frontier of agent memory, 2026. URL https://github.com/mem0ai/dolphinbench/tree/81cb6f8405b40a9e76089cef650806a80af06ea2. Repository pinned at 81cb6f8405b40a9e76089cef650806a80af06ea2.

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 2020. URL https://arxiv.org/abs/2005.11401.

[7] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023. URL https://arxiv.org/abs/2307.03172.

[8] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024. URL https://arxiv.org/abs/2402.17753.

[9] Mem0 contributors. Mem0 oss pinned source metadata: Version 2.1.0, 2026. URL https://github.com/mem0ai/mem0/blob/d3891e48baa2c6e769f9cfa4003873bd6a85bc07/pyproject.toml. Source revision d3891e48baa2c6e769f9cfa4003873bd6a85bc07; distinct from historical AGMI version 2.0.20.

[10] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14:265–294, 1978. doi: 10.1007/BF01588971.

[11] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

[12] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023. URL https://arxiv.org/abs/2304.03442.

[13] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 1999. URL https://www.schneier.com/academic/archives/1999/05/secure_audit_logs_to.html.

[14] Javad Taghia. AtMem 2.3.8: Release notes and implementation, 2026. URL https://github.com/aetna000/atmem/tree/v2.3.8. Stable source commit 267c5b3ac08e8cac26dfee13c6eac2842d4bb876.

[15] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024. URL https://arxiv.org/abs/2410.10813.

[16] Di Wu, Zixiang Ji, Asmi Kawatkar, Bryan Kwan, Jia-Chen Gu, Nanyun Peng, and Kai-Wei Chang. LongMemEval-V2: Evaluating long-term agent memory toward experienced colleagues. *arXiv preprint arXiv:2605.12493*, 2026. URL https://arxiv.org/abs/2605.12493.

A Reproducibility and Source-to-Claim Map

The supplied source package includes the LaTeX manuscript, bibliography, original vector figures and PNG exports, a figure-generation script, descriptive-statistics calculations, and compact repository evidence. Compile with `tectonic atmem-2.3.8.tex`; the README gives the exact commands. The bundle contains no API keys, private histories, or fabricated raw cases.

The benchmark summary's top-level candidate identity is not a per-arm environment lock. The paper therefore does not infer an exact LongMem source commit from the final Dolphin commit. Similarly, the Dolphin summary contains a candidate-artifact hash and the report lists a wheel hash; these are retained as named fields without assuming they are the same object. Independent reproduction needs the underlying per-arm installed-identity receipt, dependency lock, checkpoint digest, provider settings, prompts, case traces and grading outputs.

B Evidence Inventory and Identity Boundaries

Hashes identify retained artifacts; they do not prove correctness of their contents. This paper checks the compact inventory and code correspondence but does not claim access to all hashed raw files.

C Algorithmic Specification for Review

The following pseudocode summarizes the intended boundary. It is an explanatory specification, not a line-for-line implementation or a claim that the grounding relation is complete.

```
prepare_context(request):
    manifest = authorize(request.scope, request.
        generation)
    plan = derive_bounded_obligations(request.query)
    candidates = nominate_by_obligation_and_pool(plan,
         manifest)
    candidates =
        deduplicate_and_expand_source_neighbours(
        candidates)
    decision = assess_declared_evidence_coverage(plan,
         candidates)
    packed = pack_complementary_evidence(candidates,
        request.budget)
    if required_support_lost(decision, packed):
        decision = partial(reason="
            context_budget_exhausted")
    assert packed.ids subset_of manifest.
        authorized_ids
    canonical = reload_exact_selection(packed.ids)
    assert scopes_generations_lifecycles_match(
        canonical, request)
    context = serialize(canonical) if decision.
        sufficient else ""
    assert utf8_bytes(context) <= request.
        context_byte_budget
    receipt = audit(request, plan, decision, packed.
        ids, hash(context))
    return context, decision, receipt
```

D An Ablation Matrix That Would Test the Novelty Claim

These experiments should report success, incorrect action, safe block, false block, finalization failure, provider failure, and policy rejection separately. Releasing these outcomes and the exact paired cases would make the architectural contribution testable rather than dependent on aggregate launch claims.

E Exploratory Local Comparator Runs

The following rows are preserved because omitting completed runs would create a selective record. They are not headline results and do not define a product ranking.

Claim / mechanism	Source location at tag `v2.3.8`, unless noted
Legacy scoped serialization	`atmem/memory.py`, `prepare_context_v1`; also inspect tag `v2.3.7`
Legacy audit-only comparison	`atmem/memory.py`, `verify` at `v2.3.7`
Typed contracts and loss receipts	`atmem/context_engine/contracts.py` and `formation.py` in that directory
Query obligations and facets	`atmem/context_engine/planner.py`
Candidate budgets and neighbours	`atmem/context_engine/pools.py`; `atmem/context_engine/retrieval.py`
Complementary / episode packing	`atmem/context_engine/packing.py`
Status and final validation	`atmem/context_engine/sufficiency.py`; `atmem/context_engine/service.py`
Audit binding and exceptions	`atmem/memory.py`, `_record_integrity_sha256`, `_verify_record_integrity`, `verify`
LongMem Mem0 configuration	`research/production_benchmarks/adapters/longmemeval_mem0.py`
LongMem formation / embeddings	`research/production_benchmarks/prepare_longmem_memories.py`; `local_embedding_proxy.py` in that directory
Dolphin Mem0 configuration	`research/production_benchmarks/dolphinbench.py`, `Mem0DolphinAdapter`
Frozen utility summary	`benchmarks/retrieval_quality/baselines/five-percent-results-20261010.json`
Final Dolphin summary	`benchmarks/retrieval_quality/baselines/dolphinbench-final-b97d35e.json`
LongMem confirmation split	`benchmarks/retrieval_quality/protocols/longmemeval-v2-question-split-v1.json`
Dolphin confirmation split	`benchmarks/retrieval_quality/protocols/dolphinbench-task-split-v1.json`
AGMI installed artifact	`benchmarks/retrieval_quality/baselines/agmi-2.3.8-20261010.json`

Table 6: Source-to-claim map for independent inspection.

Artifact	Recorded identity
Stable architecture commit	`267c5b3ac08e8cac26dfee13c6eac2842d4bb876`
2.3.7 source commit	`cb1cf0fc6e3e12dd0a52140bc0eda4797d2e887b`
Final Dolphin candidate	`b97d35e1df7ee6ef435928bafb8189ac99a1856d`
AGMI vendor-candidate source	`fdc63de4b5ced0029f2717ee542b9600730ce3fd`
AGMI independent confirmation	AGMI PR 7
LongMem aggregate SHA-256	`89c800219cfab38b34cfc406d764919b49c24b550827a30414cb49fb4c247fd8`
Verified-evidence control SHA-256	`26ba0c7d60b950ec0a5435813b6c7316a0b0badcd9050478b1ff0f262d3bddd6`
Final Dolphin AtMem evaluation SHA-256	`aa5242c509f4748a4ab067a56e8bedda3787abfdbc6e0df0ce63b80e936fd358`
Final Dolphin Mem0 evaluation SHA-256	`2283f4b6c2c3ded66005171abeed395c2614378f3445af1ef656d259afbb712b`
AGMI vendor-candidate wheel SHA-256	`61cf17963177bd7e9c6a5e9fe18a3be30356cc0b2d4d6d373b215a9fe12266dd`
AGMI published PyPI wheel SHA-256	`05ca2c579be1af55de1da056ae257062417831b2e8eb4f2fda2a6413ba3ee923`
AGMI runner output SHA-256	`5498bedba335d6c7f40846e17e13413e22d25c298675aa34536c80315f60cceb`

Table 7: Artifact identities and digests. Hashes identify retained objects; they do not prove correctness.

Intervention	Hold fixed	Question answered
Remove typed views	Sources, learned embedder, reader, byte budget	Do evidence shapes help beyond raw source retrieval?
Remove obligation routing	Candidates and formation	Does requirement decomposition improve complementary recall?
Remove neighbours	Head ranking and budget	Do conditions and outcomes require adjacent evidence?
Remove episode grouping	Exact selected spans	Does arrangement improve reader/action use?
Disable sufficiency gate	Retrieved package and task	What is the utility versus false-blocking trade-off?
Disable audit enforcement under attacks	Seeded store and attack sequence	Does integrity checking prevent unsafe memory use?

Table 8: Proposed experiments, not completed results. Use untouched tasks and symmetric repetitions before assigning causal credit.

Mem0 uses deterministic hash embeddings with inference disabled; AgentRunbook-R uses the recorded local route; no AgentRunbook-R Dolphin result exists.

F Frozen Confirmation Protocol

This appendix records the protocol that must be executed before any comparative or generalization claim replaces the formative language in this paper. It reports no outcome.

Metric	AtMem candidate	AgentRunbook-R	Mem0 OSS
LongMem correct / 23	5 (21.7%)	4 (17.4%)	1 (4.3%)
LongMem system failures	6	4	1
Dolphin tasks / 30	9 (30.0%)	Not run	7 (23.3%)
Dolphin checks / 97	32 (33.0%)	Not run	27 (27.8%)
Dolphin system failures	0	Not run	0

Table 9: Configuration-specific formative observations. These inspected samples and constrained arms cannot support generalization, significance, or strongest-system claims.

Partitions and exclusion. Use all 318 LongMemEval-V2 IDs in the pre-existing confirmation partition of `longmemeval-v2-question-split-v1.json` and all 582 DolphinBench IDs in the pre-existing confirmation partition of `dolphinbench-task-split-v1.json`. Exclude no case after opening an outcome. Infrastructure failures remain in the primary denominator and are also reported by failure class. If feasibility requires a smaller first-stage sample, its IDs must be selected by a published salted hash before any question, answer, evidence, trace or score is inspected; the remaining IDs stay sealed.

Frozen arms. Evaluate (i) AtMem 2.3.8 at the exact stable wheel and profile, (ii) Mem0 OSS with its recommended learned embedding and normal extraction, (iii) AgentRunbook-R with its recommended formation and retrieval configuration, (iv) resource-matched versions of all three under a shared local budget, and (v) no-retrieval and evaluator-verified-evidence controls where the benchmark defines them. A missing adapter is a missing result, never a zero. Model, revision, provider, prompt, context ceiling, retries, timeouts, hardware, dependency lock and monetary budget are fixed before execution.

Primary endpoints. LongMem's primary endpoint is question correctness over confirmation IDs. Dolphin's primary endpoint is complete task pass over confirmation IDs. Secondary endpoints are evidence-set recall, exact-value retention, check success, incorrect action, safe block, false block, finalization failure, provider failure, policy rejection, latency, tokens and cost. Check-level Dolphin outcomes are clustered within tasks and are never treated as independent samples.

Repetition and inference. Run three reader repetitions per case using identical seeds where the provider exposes them and identical repetition counts otherwise. The unit of paired inference is the question or task. Compare primary endpoints with paired exact tests and report paired bootstrap intervals over cases. Control the family-wise error rate across preregistered primary system contrasts using Holm correction. Report absolute outcomes and discordant pairs, not only relative improvements. No tuning, prompt change, retry-rule change or component selection may use confirmation outcomes.

Ablations and integrity. Run the interventions in the preceding ablation matrix against the frozen AtMem artifact. Treat them as secondary analyses and report every attempted component removal. Separately rerun AGMI from a clean environment by a party unaffiliated with AtMem, covering fresh records, legacy records, legitimate lifecycle changes followed by tampering, stale external checkpoints and host enforcement. Audit detection and read blocking remain separate endpoints.

Release rule. Before opening outcomes, publish the selected IDs, configuration manifests, container or lockfile digests, prompts, analysis script and empty result schema. After execution, publish case-level stage ledgers and signed aggregate receipts. Any post-outcome repair begins a new iteration cycle and requires a still-unopened confirmation partition for subsequent claims.